Does Core Pinning Isolate the Safety Function on a Single-Chip Wi-Fi SoC? An Attack-Resilient Intrusion Alarm, Measured and Verified

MD Humayun
Department of Computer Science and Engineering
Sardar Vallabhbhai National Institute of Technology, Surat, India
github.com/mdhumayun7/Smart-Home-Automation

Abstract

Consumer intrusion alarms are built almost entirely on single-chip Wi-Fi systems-on-chip, in which the radio stack, the network stack and the application share one die and one scheduler. The real-time-security literature shows that on Linux-class industrial devices, pinning the control task to a separate core keeps its timing stable under wired network floods; whether that result transfers to a single-chip Wi-Fi SoC is only partly known: Behnke *et al.* measured 2–4 % critical-task cycle loss on an ESP32 under IP-layer load even with the network server on the other core, but did not evaluate pinning as a *mitigation*, used no adversarial model, and never exercised the 802.11 management path that lies below the IP stack. We study an ESP32 intrusion alarm whose safety loop (sensing, state machine, siren) is pinned to the application core and whose network task runs on the other core behind bounded, non-blocking queues. We contribute: (i) a dual-core firmware with authenticated commands (HMAC-SHA256 over an NVS-persisted monotonic counter), passive deauthentication-flood detection and an out-of-band signed fallback alert; (ii) two machine-checked TLA+ models proving that no network behaviour can disarm the system or block a safety step, over their full state spaces; (iii) a reproducible, seeded adversarial campaign of 6,000 trials in which command injection up to 20,000 msg/s never disarms the device and, with authentication enabled, never prevents a legitimate command either, while the baseline is armed by an attacker in every flooded trial; and (iv) a fuzzing campaign of the command parser under AddressSanitizer that finds no memory-safety or authentication defect. The remaining claim—that the physical sensor-to-siren latency is invariant to on-chip radio stress—can only be established on hardware; we specify the measurement apparatus and leave the RF campaign as the one open step. The firmware, specs, experiments and this paper are a public artifact.

1 Introduction

A networked intrusion alarm has an unusual priority order. Confidentiality matters, but the property that defines the device is *availability of the safety function*: when a sensor fires, the siren must sound within a bounded time, and it must do so *even while the device itself is under attack*, because an intruder has every incentive to attack it.

Almost all such consumer devices are built on single-chip Wi-Fi SoCs of the ESP32 class, where the Wi-Fi driver, the TCP/IP stack, the MQTT client and the application all execute on the same chip under one RTOS scheduler. This differs structurally from the industrial devices studied in the real-time-security literature, where the control task can be placed on a physically separate core or co-processor. Fischer *et al.* [1] show that on a Linux-class board, pinning the control task to a dedicated core holds its 1 ms cycle jitter essentially constant under wired SYN and ARP floods, and that only a separate co-processor is acceptable for hard real-time. The natural question for the devices people actually deploy is whether the same advice—pin the safety task to a core—delivers the same isolation when the *attacked resource is the on-chip radio driver*, which shares the die and the interrupt controller with the safety task.

We take an ESP32 intrusion alarm as the object of study. Its safety loop is pinned to the application core; Wi-Fi, time sync and MQTT run in a task on the other core and communicate with the safety loop only through two bounded FreeRTOS queues and a mutex-guarded status snapshot that the safety loop never waits on. On top of this structure we add the security mechanisms a deployable alarm needs and then ask, with formal methods, simulation and a specified hardware protocol, whether the isolation actually holds.

Contributions.

1. An open dual-core alarm firmware with authenticated commands (HMAC-SHA256 over a monotonic, reboot-persistent counter), passive deauth/disassoc flood detection, a signed out-of-band fallback alert over ESP-NOW, and optional PIR+mmWave sensor fusion (Section 3).
2. Two TLA+ specifications, checked exhaustively by TLC: one of the alarm state machine and command path, one of the safety/network isolation, each mutation-tested to show the properties are not vacuous (Section 4).
3. A seeded, reproducible adversarial campaign (6,000 trials) and a sanitizer-backed fuzzing campaign of the command parser (Section 5).
4. An ablation over four command-path designs under one adversary—no defence, a token-bucket rate limit, a static shared token, and the signed-counter path—showing that availability and integrity are separable: a rate limit cannot distinguish a legitimate command from a flood and so drops both while leaving integrity

untouched, and a static token restores availability but is defeated by capture-and-replay (Section 5).

5. A parameter sweep of the deauthentication detector which found the originally chosen threshold to be wrong: at a threshold of 8 it fired spuriously in 79.9 % of benign ten-minute runs at 60 management frames per minute. The default is now 12, at 0.25 % (Section 5).

6. A measurement protocol and instrumentation for the one claim that requires silicon—the physical latency invariance under on-chip radio stress—stated honestly as the open step (Section 6).

2 Related Work

The closest work, and what it settles. Behnke *et al.* [15] measured a periodic critical task's lateness on an *ESP32* under IP load up to 120,000 pps bursts with the network server on the other core, and found the task lost 2.15–3.8 % of its cycles (3.17–6.67 % for packets to an unused port), with lateness growing roughly linearly once the MCU saturates. The question of whether on-chip network activity reaches a critical task is therefore answered: it does, even with cross-core separation. Their follow-up work proposes mitigations, but neither is available to a developer shipping stock hardware: a priority-aware multiqueue NIC [16] that was simulated and never built, and driver-level early demultiplexing [17] prototyped on a Zynq-7000. Critically, [17] states that the ESP32's second core is used to separate observed processes from the test harness, *not* as a mitigation; core pinning is nowhere evaluated as a defence. All of this work uses IP-layer load, and none models an adversary. Three gaps therefore remain, and they define this paper: pinning plus queue discipline has never been evaluated as a mitigation; 802.11 management frames are handled below the IP stack and have never been used as the stressor; and no adversarial or formal treatment exists.

Timing isolation under attack and interference elsewhere. The consistent finding is that spatial separation plus a resource regulator preserves real-time behaviour. Its consistent platform, however, is large. Fischer *et al.* [1] measure control-task jitter on Linux-class industrial devices under *wired* floods and find core pinning effective, with a separate co-processor best; the attacked interface there is a physically separate NIC. Meszlényi *et al.* [6] hold DDS latency and jitter predictable under heavy interference using priority-based middleware scheduling, AF_XDP kernel bypass and dedicated NIC hardware queues on x86 with a PREEMPT-RT kernel. Hoornaert *et al.* [7] co-design CPU-time and memory-bandwidth isolation on a Cortex-A53 MPSoC, switching the interconnect from fair to fixed-priority arbitration on overload, using a hypervisor and an FPGA-hosted memory scheduler. The mixed-criticality literature these sit in is surveyed by Burns and Davis [5], which treats criticality as a scheduling problem under benign timing assumptions. None of the mechanisms these works rely on—an MMU, a hypervisor, performance-counter regulation, separate NIC queues—exist on a 240 MHz single-chip Wi-Fi SoC. What remains available there is core pinning and queue discipline alone, and whether that residual mechanism suffices is an open empirical question about a particular silicon and radio driver.

Table 1: Positioning. Behnke *et al.* already measured deadline degradation on this chip family, so the open question is not measurement but whether the only mitigation available on stock hardware works, under load that reaches the radio rather than the IP stack.

Work	Platform	Dead-line	Radio load	Pinning as mitigation
Behnke [15]	ESP32	yes	no	no
Behnke [16]	sim. NIC	yes	no	no
Behnke [17]	Zynq-7000	yes	no	no
Fischer [1]	Linux board	yes	no	no
Meszlényi [6]	x86 RT	yes	no	no
Hoornaert [7]	A53 + hyperv.	yes	no	no
Carbajal [2]	Wi-Fi testbed	no	yes	no
Kremser [8]	ESP32-S3	no	yes	no
Cayre [9]	ESP32 family	no	yes	no
This work	**ESP32**	**pend.**[†]	**yes**	**yes**

[†]Logical isolation is established here by model checking and simulation. The physical deadline measurement is specified but *not yet run* (Section 6), and no latency figure is claimed anywhere in this paper.

Radio-layer attacks on this device class. The attack side is established and specifically demonstrated on this chip family. Carbajal and Jodeiri Akbarfam [2] quantify deauthentication resilience across open, WPA2, PMF and WPA3 configurations, finding PMF effective under the conditions tested; the dependent variable is whether the *link* drops. Kremser and Graffi [8] build deauthentication firmware for the ESP32-S3 itself—patching the compiled image because the ESP-IDF transmit path rejects such frames—showing the attack is cheap and portable against exactly this hardware. Cayre *et al.* [9] reverse-engineer the ESP32 family's wireless ROM components and inject, alter and jam link-layer traffic from software, supporting our assumption that radio-layer stress is a realistic in-scope condition. Crucially, all of this measures the link or the attacker's reach, not whether the victim's *application* still meets its deadline.

ESP32 application-stack security. Work on this platform measures the latency cost of MQTT-over-TLS on an ESP32-C3 [3] and runs on-device ML intrusion detection on an ESP32 [4]. Both are valuable, and both optimise transport cost or detection accuracy rather than treating a safety deadline as the dependent variable; an on-device detector is itself additional load on the contended chip.

Formal verification and fuzzing. Krichen's survey of formal verification for IoT [10] covers model checking, abstract interpretation and deductive verification with Event-B, Maude and Simulink tooling, and does not record TLA+ [11] use for this device class—still less a TLA+ model of a task-isolation argument. On the fuzzing side [12], the frontier is whole-firmware: Bley *et al.* [13] wrap fuzz inputs in protocol-valid packets to reach deep network-stack code, and SysFuSS [14] adds selective symbolic execution when coverage stalls. Our fuzzing is deliberately narrower—the application-layer command parser in a host harness—and is evidence for one property with a precise oracle, not a contribution to fuzzing technique.

3 System Design

Dual-core structure. The Arduino `loop` runs the safety function on the application core: input conditioning (debouncing, a PIR warm-up and 150 ms confirmation filter), a five-state arm/disarm machine (exit and entry delays, an alarm timeout), and the outputs (siren, LEDs, display). A separate task, pinned to the other core, runs Wi-Fi with retry, NTP, the MQTT client, and event delivery. The two communicate only through a 16-deep event queue (safety→network) and a 4-deep command queue (network→safety), plus a status snapshot the safety loop copies under a zero-timeout mutex—so the safety loop never blocks on the network side. The enqueue of an event never blocks either: on a full queue the oldest is dropped and counted, never silently lost.

Authenticated commands. A command on the MQTT topic is trusted only if it carries a valid HMAC-SHA256 tag over the string `VERB:counter`, where the counter must strictly exceed the highest previously accepted value. The counter is persisted in NVS, so replay protection survives a reboot. This defeats both forgery (no key, no tag) and replay (no fresh counter), and lets a signed command safely perform remote disarm, which is otherwise refused.

Attack detection and fallback. A passive promiscuous-mode monitor counts deauthentication/disassociation frames and raises a `wifi_deauth` event when a burst exceeds a threshold; loss of the link while armed is itself raised as `link_lost`. When MQTT is unreachable, critical events are additionally sent as HMAC-signed, counter-protected ESP-NOW messages to a second node, which is peer-to-peer and so survives an attack aimed at the access point. These mechanisms add roughly 3 KB of flash and 170 B of RAM over the unhardened build.

4 Formal Verification

We model two properties in TLA+ and check them exhaustively with TLC.

State machine and command path. The first spec models the five states and the command rules. TLC verifies, over all 25 reachable states, that (i) no unauthenticated remote command ever disarms the system, (ii) the replay counter never decreases, (iii) the siren is never active while disarmed, and (iv) the entry delay and the alarm always resolve (no stuck state).

Isolation. The second spec models the two tasks with the network task made maximally adversarial: it may stall for an unbounded time, be flooded, fail to publish, and crash and restart, and it is given *no* fairness. The safety task is a loop. TLC verifies, over all 660 reachable states, that in every state the safety step is enabled (never blocked by the network side), that the network task never writes the alarm state, that the event queue never exceeds capacity, and that no produced event is lost unaccounted-for. Each property was mutation-tested: injecting the corresponding defect (e.g. a synchronous network call in the safety loop) makes TLC report the expected violation, so the properties have teeth.

Abstraction boundary. We state plainly what these models do *not* cover, because it is exactly where the research question lives. The network task is abstracted to a small set of discrete actions (go-busy, stall, crash, deliver, publish-fail); it is not a source of *timing interference*. TLC therefore establishes that, within this abstraction, no control- or data-flow dependency can block the safety step. It says nothing about contention for shared silicon—bus, cache, interrupt controller—which is a property of the implementation and not of the model. Model checking here bounds the *logical* design space; it does not substitute for measurement.

5 Simulation and Fuzzing

Adversarial campaign. We compile the unchanged firmware against host models of the hardware, network, NVS and radio, and drive it through seeded trials. In each trial a continuous attacker injects commands on the command topic—forged tags, replays of captured valid messages, unsigned commands and random bytes—at a controlled rate while a legitimate user arms the system and an intrusion then occurs. We ran 3,000 trials each for the baseline (no command authentication) and hardened (authentication on) builds, across injection rates from 0 to 20,000 msg/s, every trial reproducible from its seed.

Table 2 and Fig. 1 give the result. In the baseline, an attacker changed the system state (a nuisance arm) in all 2,500 flooded trials; with authentication, in none. With authentication, a legitimate command took effect in all 3,000 trials regardless of attack rate; in the baseline, flooding starved the legitimate command in every flooded trial. No attacker ever *disarmed* the device in any of the 6,000 trials, and no safety invariant was violated. Because the simulated clock is advanced by the harness, the safety latency has zero variance here (a deterministic 151 ms, the confirmation window plus loop period); this establishes the *logical* invariance—no code path makes the safety response depend on network input—and is why the *timing* invariance is deferred to hardware (Section 6). For the same reason we report no hypothesis test on these data: with zero variance any test is trivially satisfied and carries no information. The analysis that *will* be applied to the hardware latencies—a two-sided Mann-Whitney U test with tie correction, implemented in the committed artifact—is fixed in advance here, so it is pre-registered rather than chosen after seeing the measurements.

What the campaign does and does not show. The security outcome—that forged commands cannot arm or disarm an authenticated build—follows deductively from the construction, and 6,000 trials do not make it more true; the campaign's role there is to validate that the *implementation* matches the design. The informative result is the availability one, which does not follow from the cryptography: under flood the baseline loses the legitimate command in every trial while the hardened build retains it in all 3,000, because rejection happens on the network core before the bounded queue, so a flood cannot displace a genuine command. The baseline is also a weak adversary by construction (no authentication at all); comparison against stronger alternatives—transport security with broker credentials, link-layer protected management frames, or rate limiting—is left to future work.

Table 2: Adversarial campaign, 3,000 trials per build. "Attacker armed" and "disarmed" count trials where an *unauthenticated* command changed state.

Build	Attacker armed	Attacker disarmed	Legit cmd succeeded	Invariant violations
Baseline	2500/3000	0	500/3000	0
Hardened	0/3000	0	3000/3000	0

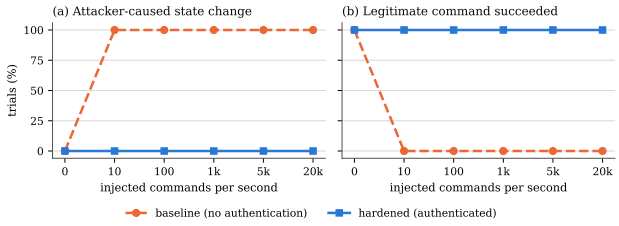


Figure 1: Effect of command-injection rate. Left: fraction of trials in which the attacker changed the system state. Right: fraction in which the legitimate command took effect. The hardened build is flat across five orders of magnitude of attack intensity.

Fuzzing. The command parser is the one input a network peer fully controls. We fuzz it with the firmware built under AddressSanitizer and UndefinedBehaviorSanitizer, feeding random bytes, malformed and overlong fields, non-decimal and overflowing counters, embedded NULs, and bit-flip mutations of a near-miss signed message. After every input we check memory safety and the invariant that no input lacking a valid tag changes state or advances the counter. Across 3,000,000 iterations the fuzzer found no memory error and no accepted input. We read this as evidence of *memory safety and parser robustness*: with a 128-bit tag, zero forgeries is the expected outcome and is not independent evidence that authentication is sound. One episode is worth recording as a methodological caution. An early run reported an accepted input; the mutation had flipped a hex digit's case, which decodes to identical tag bytes, so the "finding" was the genuine command and acceptance was correct. The corpus now signs its mutation seed with a deliberately wrong key so no mutation can yield a valid command by accident.

Ablation over command-path designs. To avoid resting on a baseline that is trivially defeated, we ran the same adversary against four designs: no defence, a token-bucket rate limit (5 commands/s), a static shared token in the payload, and the signed-counter path. The rate limit and the token are modelled as edge filters, which is where a developer would place them. 300 trials each, same seeds. Table 3 gives the result, and it separates two properties that are easily conflated. A rate limiter cannot distinguish a legitimate command from a flood, so it drops both and leaves integrity entirely untouched. A static token *does* restore availability, because the edge filters the flood cheaply—but it travels in plaintext, so one captured command defeats it. Only the monotonic counter stops replay; a MAC alone would behave like the token row. The alarm fired in every trial of every design, which is the isolation property doing its job: the safety function does not depend on the command path.

Table 3: Command-path ablation, 300 trials per design, identical adversary and seeds. "Survives flood" counts flooded trials in which a legitimate command took effect; "replayed" counts trials in which the attacker obtained an accepted command using only captured or forged material.

Design	Survives flood	Defeated by replay	Alarm fired
No defence	0/250	250/300	300/300
Rate limit (5/s)	0/250	246/300	300/300
Static shared token	250/250	250/300	300/300
HMAC + counter	**250/250**	**0/300**	**300/300**

Detector parameter sensitivity, and a defect it found. The deauthentication detector's threshold and window had a stated rationale but no measurement. Sweeping them against a Poisson model of benign management-frame arrivals (2,000 runs of ten simulated minutes per grid cell) showed the original threshold of 8 was wrong: at 60 frames/min, an ordinary busy multi-AP environment, it fired spuriously in 79.9 % of runs. A threshold of 12 gives 0.25 % while still detecting every swept attack rate with no misses, at a cost of about 214 ms of detection latency at 20 frames/s. We changed the default on that evidence. We stress that the benign arrival model is an *assumption*: no capture of real management traffic was available, so this characterises the detector's logic rather than any real network, and a measured rate remains future work.

6 The Open Step: Hardware Latency Invariance

The campaign establishes logical invariance; it cannot establish *timing* invariance, because the simulated clock does not model CPU or interrupt contention. That is precisely the property in question on a single-chip SoC, and it can only be measured on silicon. We specify the apparatus: an independent ESP32 times the interval from the PIR line to the siren line with its own hardware timer (so the measuring clock is never the clock under load), across four conditions—idle, heavy legitimate traffic, command-interface stress, and radio-layer stress in an authorized isolated environment—and the same non-parametric test compares each loaded condition against idle. A null result confirms that core pinning isolates the safety function on this class of device, extending [1] to single-chip Wi-Fi SoCs; a positive result would show it does *not*, directly challenging standard practice. We report the protocol rather than a number, and do not speculate on the outcome.

7 Threats to Validity and Ethics

Simulation shows logical, not physical, invariance (addressed by Section 6). Results are for one SoC family and one firmware; generalisation is a stated limitation. Latency must be measured externally, not by the device under load. All adversarial radio work is confined to an owned, isolated environment with permission; no third-party network is targeted, and this paper deliberately omits any attack-generation procedure.

8 Conclusion

We cast a consumer intrusion alarm as a safety-critical system and asked whether the standard single-chip mitigation—pin the safety task to a core—actually isolates it from attacks on the device's own radio and command interface. We answer the logical half of that question with exhaustive model checking and a 6,000-trial reproducible campaign, add the command authentication, attack detection and fallback a deployable alarm needs, and specify the one hardware measurement that settles the timing half. The artifact is public so the result can be reproduced and the hardware step completed. All claims are scoped to the platform actually tested—the dual-core Xtensa ESP32 under arduino-esp32 2.0.17 and 3.3.12—and we do not generalise to single-chip Wi-Fi SoCs as a class. The single-core variants of the same family are the more interesting open case, since they cannot pin at all.

Declarations

Funding. This research received no external funding.

Conflicts of interest. The author declares no conflict of interest.

Author contributions. MD Humayun: conceptualisation, methodology, software, formal analysis, investigation, data curation, writing (original draft, review and editing), visualisation.

Data availability. All data reported in this paper are generated deterministically from fixed seeds by the scripts in the public repository; the aggregate results files are committed, and the full per-trial records regenerate identically by running `experiments/run.sh`. No third-party or human-subject data were used.

Code availability. Firmware, formal specifications, experiment harnesses, analysis scripts and figure sources are public at `github.com/mdhumayun7/Smart-Home-Automation` under the MIT licence. Continuous integration re-runs the formal verification and a reduced adversarial campaign on every commit.

Ethics. No human or animal subjects were involved. No third-party network, device or system was attacked or tested. The repository deliberately publishes no attack-generation procedure or tooling, and the radio-layer condition described in Section 6 is specified for execution only on equipment owned by the experimenter, in an isolated environment, with institutional permission.

Generative AI disclosure. An AI assistant was used for drafting assistance, documentation and code review. All research claims, experimental designs, results and conclusions are the author's own; every result reported here was produced by the committed scripts and verified by the author, and every reference was checked against its publisher or preprint record.

References

[1] F. Fischer *et al.*, "Analysis of Industrial Device Architectures for Real-Time Operations under Denial of Service Attacks," arXiv:2007.08885, 2020.

[2] A. Carbajal and A. Jodeiri Akbarfam, "A Software-Defined Testbed for Quantifying Deauthentication Resilience in Modern Wi-Fi Networks," arXiv:2602.23513, 2026.

[3] M. A. Azhar, M. Azwar, and O. Asroni, "Performance and Security Analysis of MQTT over TLS on ESP32-Based IoT Systems," *J. Applied Informatics and Computing*, 2026.

[4] V. Ieropoulos *et al.*, "Empowering IoT Security: On-Device Intrusion Detection in Resource Constrained Devices," arXiv:2605.13159, 2026.

[5] A. Burns and R. I. Davis, "Mixed Criticality Systems: A Review," 14th edn., Dept. of Computer Science, University of York, Feb. 2026.

[6] L. Meszlényi, J. Kahle, D. Püllen, S. Kowalewski, S. Katzenbeisser, and A. Kampmann, "Towards Timing Isolation for Mixed-Criticality Communication in Software-Defined Vehicles," arXiv:2508.13652, 2025.

[7] D. Hoornaert, G. Ghaemi, A. Bastoni, R. Mancuso, M. Caccamo, and G. Corradi, "Mcti: mixed-criticality task-based isolation," *Real-Time Systems*, vol. 60, no. 2, pp. 328–365, 2024.

[8] S. Kremser and K. Graffi, "Evaluating the ESP32-S3 for Wi-Fi Penetration Testing Through the Development of Deauther32 and HackHeld32," *Sensors*, vol. 26, no. 11, art. 3287, 2026.

[9] R. Cayre, D. Cauquil, and A. Francillon, "ESPwn32: Hacking with ESP32 System-on-Chips," in *Proc. 17th IEEE Workshop on Offensive Technologies (WOOT)*, 2023.

[10] M. Krichen, "A Survey on Formal Verification and Validation Techniques for Internet of Things," *Applied Sciences*, vol. 13, no. 14, art. 8122, 2023.

[11] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[12] J. Yun *et al.*, "Fuzzing of Embedded Systems: A Survey," *ACM Computing Surveys*, 2022.

[13] M. Bley, T. Scharnowski, S. Wörner, M. Schloegel, and T. Holz, "Protocol-Aware Firmware Rehosting for Effective Fuzzing of Embedded Network Stacks," ACM, 2025, doi:10.1145/3719027.3765125.

[14] D. Tharindu, A. Jayasena, and P. Mishra, "SysFuSS: System-Level Firmware Fuzzing with Selective Symbolic Execution," arXiv:2602.02243, 2026.

[15] I. Behnke, L. Pirl, L. Thamsen, R. Danicki, A. Polze, and O. Kao, "Interrupting Real-Time IoT Tasks: How Bad Can It Be to Connect Your Critical Embedded System to the Internet?," in *Proc. IEEE 39th Int. Performance Computing and Communications Conf. (IPCCC)*, 2020.

[16] I. Behnke, P. Wiesner, R. Danicki, and L. Thamsen, "A Priority-Aware Multiqueue NIC Design for Real-Time IoT Devices," in *Proc. 37th ACM/SIGAPP Symp. on Applied Computing (SAC)*, 2022, doi:10.1145/3477314.3507165.

[17] I. Behnke, C. Blumschein, R. Danicki, P. Wiesner, L. Thamsen, and O. Kao, "Towards a Real-Time IoT: Approaches for Incoming Packet Processing in Cyber-Physical Systems," *Journal of Systems Architecture*, vol. 140, art. 102891, 2023, doi:10.1016/j.sysarc.2023.102891.